Entregável 2 explicado

Rede de distribuição: fechamento e proximidade

Algoritmos e Estruturas de Dados – BCC – 2026.2

Laura da Costa Corte Cordeiro e Ana Helena de Souza Macedo

O que este documento cobre

- Os assuntos cobrados no Entregável 2: fila de prioridade com alteração de chave e divisão e conquista.
- A lógica do arquivo `e2.py`: o heap indexado, a rede, o REMOVER e o PAR, com os trechos do próprio arquivo.
- Uma legenda com todas as notações, simulações passo a passo do heap e do PAR, complexidades e perguntas da entrevista.
- As partes herdadas do E1 (Kadane e pilha) continuam iguais e estão explicadas no documento do Entregável 1.

Sumário

- 1. Legenda das notações
- 2. O que o Entregável 2 pede
- 3. Assuntos que caem
- 4. Lógica do código: o heap
- 5. Lógica do código: a rede e o REMOVER
- 6. Lógica do código: o PAR
- 7. Mudanças no main
- 8. Complexidades
- 9. Armadilhas comuns
- 10. Perguntas prováveis da entrevista

1. Legenda das notações

Símbolo	Significado
n	Número de depósitos vivos no momento
k (em REMOVER)	Número de depósitos removidos por uma chamada de REMOVER
k (período)	Número do período, como no E1
T	Tolerância mínima: REMOVER fecha quem tem lucro **estritamente menor** que T
P, h, L_k	Como no E1: períodos, altura da pilha, lucro do período k
d2	Quadrado da distância entre dois pontos: $(x_a - x_b)^2 + (y_a - y_b)^2$
d	Distância arredondada para cima: menor inteiro t com $t^2 >= d2$
xm	Coordenada x da linha divisória no PAR
m (em par_rec)	Tamanho do trecho pts[lo:hi] (hi − lo); não confundir com o m de rotas do enunciado
T(m)	Custo de par_rec para m pontos, na recorrência T(m) = 2T(m/2) + O(m)

Nome no código	O que guarda
`heap['ids']`	Ids na ordem do heap; ids[0] é o topo (menor chave)
`heap['chaves']`	chaves[i] é a chave (aqui, o lucro) de ids[i]
`heap['pos']`	Dicionário id : posição do id dentro de ids
`rede['lucros']`, `rede['coords']`	Lucro e coordenadas (x, y) de cada depósito vivo
`rede['fila']`	O heap de lucros
`rede['total']`	Soma dos lucros dos vivos
`rede['par_cache']`	Última resposta do PAR, ou None se precisa recalcular
`pts`	Lista de pontos (x, y, id) usada pelo PAR
`lo, hi, meio`	Limites do trecho de pts tratado e o ponto onde ele é dividido
`faixa`	Pontos perto da linha divisória, com $(x - xm)^2 < d2$

Termo	Significado
Pai / filhos	No heap guardado em lista, o pai da posição i é (i − 1) // 2; os filhos são 2i + 1 e 2i + 2
Subir / descer	Trocar um elemento com o pai (subir) ou com o menor filho (descer) até o heap voltar a valer
Propriedade de heap	Cada elemento tem chave menor ou igual à dos seus filhos; o menor fica no topo

2. O que o Entregável 2 pede

Operações exercitadas: **REMOVER** e **PAR**, com **INSERIR** e **LUCRO** para a rede mudar. Rotas continuam descartadas. O enunciado espera:

- REMOVER localizando os depósitos abaixo de T **sem examinar todos os vivos**, mesmo com lucros mudando;

- o registro do E1 passando a ter remoção;
- PAR calculado algoritmicamente, **melhor que comparar todos os pares**;
- proibido guardar distâncias entre pares (matriz, tabela, heap de pares); permitido guardar o último resultado enquanto o conjunto de vivos não muda.

3. Assuntos que caem

Tópicos do enunciado: **fila de prioridade com alteração de chave** e **divisão e conquista**. O guia acrescenta recorrências e aritmética inteira.

3.1 Fila de prioridade e heap binário

Uma fila de prioridade entrega sempre o elemento de menor (ou maior) chave. O **heap binário** é uma árvore quase completa guardada numa lista, em que cada pai tem chave menor ou igual à dos filhos. Assim o mínimo fica sempre na posição 0.

Operação	Como funciona	Custo
Ver o mínimo	Lê a posição 0	O(1)
Inserir	Coloca no fim e sobe	O(log n)
Remover o mínimo	Troca com o último, remove, e desce a nova raiz	O(log n)

A altura de um heap com n elementos é cerca de $\log_2 n$, por isso subir e descer custam O(log n).

3.2 Alteração de chave: heap indexado

O LUCRO muda o lucro de um depósito qualquer, que pode estar no meio do heap. Um heap comum não sabe onde ele está. O **heap indexado** resolve isso com o dicionário `pos` (id : posição), atualizado a cada troca. Há duas formas de lidar com chaves que mudam:

	Heap indexado (o do código)	Remoção preguiçosa
Ideia	pos acha o elemento; muda a chave e sobe ou desce	Insere um par novo a cada LUCRO e ignora pares velhos ao extrair
Custo do LUCRO	O(log n)	O(log do tamanho do heap)
Memória	No máximo n elementos	Pode acumular um par por operação
Validação	Nenhuma	Ao extrair, conferir se o par ainda é atual

3.3 Divisão e conquista

Técnica em três passos: **dividir** o problema em partes menores, **conquistar** cada parte recursivamente e **combinar** as respostas. O PAR é o exemplo clássico: comparar todos os pares custa $O(n^2)$ (cerca de 12,5 milhões de comparações para n = 5000; com 500 consultas, mais de 6 bilhões), e a divisão e conquista faz O(n log n).

3.4 Par de pontos mais próximo

- Ordena os pontos por x e divide ao meio. O par mais próximo está todo na esquerda, todo na direita, ou cruza a divisória.
- Seja d2 o menor resultado das duas metades. Um par que cruza só pode ser melhor se ambos os pontos estiverem a menos de d da divisória: é a **faixa**.

- **Argumento de empacotamento:** num retângulo d × 2d cabem poucos pontos (no máximo 8) a distância de pelo menos d entre si. Ordenando a faixa por y, cada ponto só precisa ser comparado com no máximo 7 seguintes.

3.5 Recorrências

O custo de um algoritmo recursivo é descrito por uma recorrência. Pelo Teorema Mestre ou pela árvore de recursão:

Recorrência	Resultado	Quando acontece
T(n) = 2T(n/2) + O(n)	O(n log n)	Junção por y em tempo linear (o caso do código)
T(n) = 2T(n/2) + O(n log n)	O(n $\log^2$ n)	Se a faixa fosse reordenada do zero em cada nível

Árvore de recursão: log n níveis, cada um com trabalho total O(n), logo O(n log n).

3.6 Aritmética inteira

Comparar distâncias ao quadrado evita a raiz e os erros de ponto flutuante. Para imprimir, o enunciado pede o teto da raiz: `isqrt(d2)` dá o maior t com t^2 <= d2; se t^2 < d2, soma 1.

4. Lógica do código: o heap

O heap é um dicionário com três listas/dicionários, criado por `novo_heap`. Invariantes: `pos[ids[i]] == i` para todo i, e cada pai com chave <= à dos filhos.

```
def novo_heap():
    return {"ids": [], "chaves": [], "pos": {}}
```

4.1 Subir e descer

```
def heap_subir(heap, i):
    ids, chaves, pos = heap["ids"], heap["chaves"], heap["pos"]
    ident, chave = ids[i], chaves[i]
    while i > 0:
        pai = (i - 1) // 2
        if chaves[pai] <= chave:
            break
        ids[i], chaves[i] = ids[pai], chaves[pai]
        pos[ids[i]] = i
        i = pai
    ids[i], chaves[i] = ident, chave
    pos[ident] = i
```

```
def heap_descer(heap, i):
    ids, chaves, pos = heap["ids"], heap["chaves"], heap["pos"]
    tam = len(ids)
    ident, chave = ids[i], chaves[i]
    while True:
        filho = 2 * i + 1
        if filho >= tam:
            break
        if filho + 1 < tam and chaves[filho + 1] < chaves[filho]:
            filho += 1
        if chaves[filho] >= chave:
            break
        ids[i], chaves[i] = ids[filho], chaves[filho]
        pos[ids[i]] = i
        i = filho
    ids[i], chaves[i] = ident, chave
    pos[ident] = i
```

- Em vez de trocar par a par, as funções guardam o elemento que se move (`ident, chave`), deslocam pais (ou filhos) e só escrevem o elemento no lugar final. É a mesma coisa que várias trocas, com menos escritas.
- Toda posição escrita atualiza `pos`: é isso que mantém o invariante.
- No descer, o elemento troca com o **menor** dos dois filhos; trocar com o maior quebraria a propriedade de heap.

4.2 Inserir, alterar e remover

```
def heap_inserir(heap, ident, chave):
    heap["ids"].append(ident)
    heap["chaves"].append(chave)
    heap["pos"][ident] = len(heap["ids"]) - 1
    heap_subir(heap, len(heap["ids"]) - 1)
```

```
def heap_alterar(heap, ident, chave):
    i = heap["pos"][ident]
    antiga = heap["chaves"][i]
    heap["chaves"][i] = chave
    if chave < antiga:
        heap_subir(heap, i)
    elif chave > antiga:
        heap_descer(heap, i)
```

```
def heap_remover(heap, ident):
    ids, chaves, pos = heap["ids"], heap["chaves"], heap["pos"]
    i = pos.pop(ident)
    ultimo_id = ids.pop()
    ultima_chave = chaves.pop()
    if i == len(ids):
        return
    ids[i], chaves[i] = ultimo_id, ultima_chave
    pos[ultimo_id] = i
    if i > 0 and chaves[(i - 1) // 2] > ultima_chave:
        heap_subir(heap, i)
    else:
        heap_descer(heap, i)
```

- **alterar:** se a chave diminuiu, ela só pode violar a ordem com o pai, então sobe; se aumentou, só com os filhos, então desce.

- **remover:** o último elemento ocupa o buraco. Ele pode ser menor que o novo pai (sobe) ou maior que os filhos (desce). Se o removido era o próprio último, basta retirá-lo.

4.3 Simulação: o heap no exemplo

Passo	ids	chaves	O que aconteceu
insere 1 (10)	[1]	[10]	Heap vazio: fica no topo
insere 2 (4)	[2, 1]	[4, 10]	4 < 10: sobe
insere 3 (2)	[3, 1, 2]	[2, 10, 4]	Posição 2, pai 0 (chave 4): 2 < 4, sobe
insere 4 (6)	[3, 4, 2, 1]	[2, 6, 4, 10]	Posição 3, pai 1 (10): sobe; pai 0 (2) <= 6: para
LUCRO 1 −20	[1, 3, 2, 4]	[−20, 2, 4, 6]	Chave diminuiu: sobe até o topo
REMOVER (T = 0)	[3, 4, 2]	[2, 6, 4]	Topo −20 < 0: remove o 1; o 4 vai ao topo e desce; topo 2 >= 0: para. Imprime 1

5. Lógica do código: a rede e o REMOVER

A rede é um dicionário que agrupa tudo o que muda junto quando um depósito entra ou sai.

```
def nova_rede():
    return {"lucros": {}, "coords": {}, "fila": novo_heap(),
            "total": 0, "par_cache": None}
```

Invariante da rede	Quem garante
lucros, coords e fila têm os mesmos ids	inserir_deposito e remover_deposito mexem nos três
a chave de cada id na fila é o seu lucro	mudar_lucro chama heap_alterar
total == soma dos lucros	as três funções ajustam o total
par_cache, se não for None, está certo	inserir e remover apagam o cache; LUCRO não mexe nas coordenadas

5.1 Inserir, mudar lucro e a função única de remoção

```
def inserir_deposito(rede, ident, x, y, lucro):
    rede["lucros"][ident] = lucro
    rede["coords"][ident] = (x, y)
    heap_inserir(rede["fila"], ident, lucro)
    rede["total"] += lucro
    rede["par_cache"] = None
```

```
def mudar_lucro(rede, ident, lucro):
    rede["total"] += lucro - rede["lucros"][ident]
    rede["lucros"][ident] = lucro
    heap_alterar(rede["fila"], ident, lucro)
```

```
def remover_deposito(rede, ident):
    heap_remover(rede["fila"], ident)
    rede["total"] -= rede["lucros"].pop(ident)
    del rede["coords"][ident]
    rede["par_cache"] = None
```

O guia insiste: toda remoção passa por **uma única função**. Esquecer de descontar o lucro do total, por exemplo, deixaria todos os L_k seguintes errados. No E3, a remoção das rotas entra justamente aqui.

5.2 REMOVER

```
def remover_abaixo(rede, T):
    fila = rede["fila"]
    removidos = 0
    while fila["ids"] and fila["chaves"][0] < T:
        remover_deposito(rede, fila["ids"][0])
        removidos += 1
    return removidos
```

- O menor lucro está sempre no topo. Enquanto ele for **< T**, remove. Lucro igual a T fica.
- Os demais vivos nunca são examinados: custo O((k + 1) log n), com k removidos.
- Imprime a contagem, inclusive 0. Depósitos abaixo de T só saem quando REMOVER roda; até lá, continuam entrando no total.

6. Lógica do código: o PAR

6.1 par: o resultado guardado e o arredondamento

```
def par(rede):
    if rede["par_cache"] is None:
        pts = sorted((x, y, ident) for ident, (x, y) in rede["coords"].items())
        d2, a, b = par_rec(pts, 0, len(pts))
        if a > b:
            a, b = b, a
        t = isqrt(d2)
        if t * t < d2:
            t += 1
        rede["par_cache"] = f"{a} {b} {t}"
    return rede["par_cache"]
```

- Se `par_cache` vale, devolve em O(1). Senão, monta os pontos, ordena por x (e por y no empate), chama `par_rec` e guarda a resposta.
- Os ids são trocados para sair a < b, como pede o enunciado.
- Teto da raiz com inteiros: `isqrt` e +1 se não for quadrado perfeito.

6.2 par_rec: divisão e conquista

```
def par_rec(pts, lo, hi):
    if hi - lo <= 3:
        melhor_par = None
        for i in range(lo, hi):
            xi, yi, ai = pts[i]
            for j in range(i + 1, hi):
                xj, yj, aj = pts[j]
                d2 = (xi - xj) ** 2 + (yi - yj) ** 2
                if melhor_par is None or d2 < melhor_par[0]:
                    melhor_par = (d2, ai, aj)
        pts[lo:hi] = sorted(pts[lo:hi], key=lambda p: p[1])
        return melhor_par

    meio = (lo + hi) // 2
    xm = pts[meio][0]
    esq = par_rec(pts, lo, meio)
    dir_ = par_rec(pts, meio, hi)
    melhor_par = esq if esq[0] <= dir_[0] else dir_

    pts[lo:hi] = sorted(pts[lo:hi], key=lambda p: p[1])   # junção por y
    d2_min = melhor_par[0]
    faixa = [p for p in pts[lo:hi] if (p[0] - xm) ** 2 < d2_min]
    for i in range(len(faixa)):
        xi, yi, ai = faixa[i]
        for j in range(i + 1, len(faixa)):
            xj, yj, aj = faixa[j]
            dy = yj - yi
            if dy * dy >= d2_min:
                break
            d2 = (xi - xj) ** 2 + dy * dy
            if d2 < d2_min:
                d2_min = d2
                melhor_par = (d2, ai, aj)
    return melhor_par
```

Parte do código	O que faz
hi − lo <= 3	Caso base: compara todos os pares (no máximo 3) e ordena o trecho por y
meio, xm	Divide pela **posição** (não pelo valor de x, que pode repetir) e guarda o x da divisória **antes** de a recursão reordenar o trecho
esq, dir_	Resolve as duas metades
sorted(..., key=y)	Junta as duas metades já ordenadas por y; o Timsort do Python só as intercala, em tempo linear
faixa	Pontos com $(x - xm)^2 < d2$: compara distâncias ao quadrado, sem raiz
break com dy·dy >= d2_min	Na faixa ordenada por y, os próximos só ficam mais longe: para de comparar

6.3 Simulação: o primeiro PAR do exemplo

Pontos vivos: 1 (0,0), 2 (2,2), 3 (3,4), 4 (10,10). Ordenados por x, com 4 pontos: divide em [1, 2] e [3, 4], xm = 3.

- Esquerda: par (1, 2), d2 = 4 + 4 = 8. Direita: par (3, 4), d2 = 49 + 36 = 85. Melhor: 8.
- Faixa: $(x - 3)^2 < 8$ deixa o ponto 2 (1 < 8) e o ponto 3 (0 < 8); os pontos 1 (9) e 4 (49) ficam de fora.
- Comparando 2 e 3: dy = 2, $dy^2 = 4 < 8$; d2 = 1 + 4 = 5 < 8. Novo melhor: (2, 3), d2 = 5.
- isqrt(5) = 2 e 2·2 = 4 < 5, então d = 3. Saída: **2 3 3**.

7. Mudanças no main

```
rede = nova_rede()
for _ in range(n):
    ident, x, y, lucro = (int(t) for t in tokens[pos:pos + 4])
    inserir_deposito(rede, ident, x, y, lucro)
    pos += 4
pos += 3*m
```

```
if op == "INSERIR":
    ident, x, y, lucro, r = (int(t) for t in tokens[pos:pos + 5])
    pos += 5 + 3 * r
    inserir_deposito(rede, ident, x, y, lucro)

elif op == "LUCRO":
    ident, lucro = int(tokens[pos]), int(tokens[pos+1])
    pos += 2
    mudar_lucro(rede, ident, lucro)

elif op == "MELHOR":
    saida.append(melhor(kadane))

elif op == "PROSPERIDADE":
    saida.append(prosperidade(pilha, soma_pilha))

elif op == "DIST":
    pos += 1

elif op == "REMOVER":
    saida.append(str(remover_abaixo(rede, T)))

elif op == "PAR":
    saida.append(par(rede))
```

- Tudo passa pela `rede`: se o main mantivesse um `lucros` e um `total` próprios, o heap nunca saberia dos depósitos.
- O registro dos períodos usa `rede['total']`.
- DIST não aparece nos testes do E2, mas o argumento é lido para a leitura não se perder (o exemplo do enunciado tem DIST).

8. Complexidades

Operação	Tempo	Justificativa
INSERIR	O(log n)	Inserção no heap; dicionários em O(1) esperado
LUCRO	O(log n)	Alteração de chave no heap
REMOVER	O((k + 1) log n)	k remoções de O(log n) e um teste do topo
PAR (recalculando)	O(n log n)	Ordenação mais T(n) = 2T(n/2) + O(n)
PAR (com cache)	O(1)	Devolve a resposta guardada
MELHOR, PROSPERIDADE, registro	Como no E1	O(1), O(1) e O(1) amortizado

9. Armadilhas comuns

Erros que derrubam o E2

- Esquecer de descontar o lucro do total ao remover (L_k errado depois).
- Usar <= T em vez de < T no REMOVER.
- Não atualizar o heap quando o LUCRO muda o lucro.
- Esquecer de atualizar pos em alguma troca do heap.
- Dividir os pontos pelo valor de x e cair em recursão infinita com x repetido.
- Montar a faixa com |dx| <= d (raiz) em vez de dx^2 < d2.
- Invalidar o cache do PAR no LUCRO (desnecessário) ou não invalidar no INSERIR e no REMOVER (errado).

10. Perguntas prováveis da entrevista

Pergunta	Ideia central da resposta
Heap indexado ou remoção preguiçosa? Por quê?	Indexado: memória limitada a n e nenhuma validação ao extrair; custa o dicionário pos.
Por que o REMOVER não varre todos?	O heap mantém o menor lucro no topo; remove-se enquanto o topo < T.
Para que serve o pos?	Achar a posição de um id em O(1) esperado para alterar ou remover a chave.
Explique o PAR e por que a faixa é barata.	Divide por x, resolve as metades, confere a faixa; empacotamento: no máximo 7 vizinhos por ponto.
Qual a recorrência e o resultado?	T(n) = 2T(n/2) + O(n), logo O(n log n).
Por que d2 e não a distância real?	Evita ponto flutuante; o teto é isqrt mais 1 se não for quadrado perfeito.
O cache do PAR continua válido depois de um LUCRO?	Sim: LUCRO não muda o conjunto de vivos nem as coordenadas.
Por que uma única função de remoção?	Garante que fila, registro, total e cache mudem juntos; no E3 ela ganha a parte do grafo.